

Comparative Sentiment Analysis of Political and Social Discourse Across News Media and Online Communities

Programming for data analysis

Ammi Susana Pineda Guzman - 20221020064
David Santiago Buitrago Cerquera - 20221020085
Cristhian Samuel Truque Cabrejo - 20221020071

Systems Engineering

University Distrital Francisco José de Caldas

Bogotá D.C, Colombia

GitHub Project Link: <https://github.com/SantYago710/dataAnalysis>

I. EXECUTIVE SUMMARY

This project outlines the design and implementation of a comprehensive data engineering pipeline for analysing political and social discourse sentiment in Colombia. The system integrates multiple heterogeneous data sources, including structured data obtained from the Reddit API and unstructured textual data scraped from La Silla Vacía's website.

The pipeline has a layered architecture comprising Bronze, Silver and (planned) Gold layers. Data ingestion, transformation and orchestration are managed using Apache Airflow within a Dockerised environment to ensure reproducibility and scalability.

The Bronze layer stores raw data in JSON format, preserving the original structure of the sources. The Silver layer applies a comprehensive pre-processing pipeline to transform the data into structured Parquet files optimised for analytical processing. This includes data cleaning, normalisation, outlier handling and text pre-processing techniques for natural language processing (NLP).

The system implements two functional Airflow DAGs: a Bronze ingestion DAG and a Silver processing DAG, which are triggered automatically using a FileSensor. This approach ensures traceability, idempotency and efficient data handling, providing a robust basis for future analytical and visualisation tasks in the Gold layer.

II. ARCHITECTURE DIAGRAM

This section presents an overview of the proposed data pipeline architecture. The diagram shows the main components of the system, such as the data sources, the orchestration, the storage layers and the planned analytical and visualisation stages. It provides a high-level overview of the flow of data from ingestion to processing and, ultimately, insight generation.

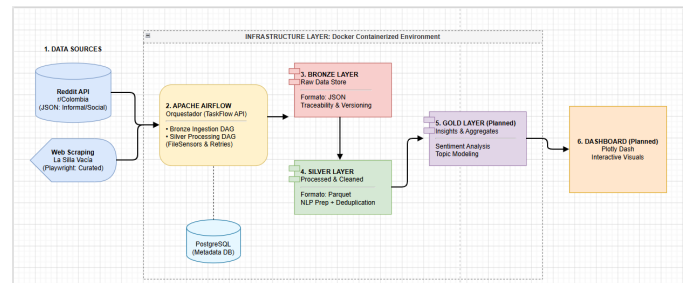


Fig. 1. Architecture Diagram of project

III. TECHNICAL ARCHITECTURE DESCRIPTION

The proposed system is based on a layered data engineering architecture that is designed to ingest, process and prepare textual data for sentiment analysis. This architecture comprises multiple interconnected components, including data sources, an orchestration layer, and storage layers (Bronze and Silver). Planned analytical and visualisation layers (Gold and Dashboard) are also included.

This clear separation of concerns ensures that each layer is responsible for a specific stage of data processing. This modular design improves the system's overall scalability, maintainability, and traceability.

A. Data Sources

The first is the Reddit API, which provides access to user-generated content from the r/Colombia subreddit. This API delivers data in JSON format, including fields such as post title, content, author, timestamp, score and URL. This data is informal and highly dynamic in nature, reflecting real-time public opinion and spontaneous discussions. This makes it particularly valuable for capturing authentic sentiment and grassroots-level discourse.

The second source is La Silla Vacía, which is accessed via a web scraping process targeting its opinion section. Unlike the

Reddit API, this source provides structured, curated content written by journalists, academics and public figures. The extracted data includes article titles, authors, publication dates, tags and full article content. Due to anti-bot protection mechanisms, the scraping process requires advanced techniques such as browser automation, user-agent spoofing and request throttling to ensure stable, reliable data extraction.

Integrating these two sources was a deliberate design decision. While Reddit provides unfiltered, large-scale public opinion, La Silla Vacía contributes high-quality, context-rich institutional discourse. Together, they enable a more comprehensive analysis by facilitating comparisons between citizen perspectives and expert narratives.

B. Orchestration Layer (Airflow)

The orchestration of the entire pipeline is managed using Apache Airflow, which plays a central role in coordinating data ingestion and processing tasks. Airflow enables the definition of workflows as Directed Acyclic Graphs (DAGs), where each node represents a task and edges define dependencies between tasks.

The Airflow Scheduler is responsible for triggering workflows based on a predefined schedule. In this project, a daily ingestion frequency is implemented, ensuring that the dataset is periodically updated without unnecessary computational overhead. This scheduling strategy aligns with the nature of the data sources, where new content is generated regularly but does not require real-time processing.

Two main DAGs are implemented in the system. The first one, known as the Bronze Ingestion DAG, is responsible for executing the data extraction process. It includes tasks that connect to the Reddit API, perform web scraping, validate the retrieved data, and store it in the Bronze layer. The second DAG, referred to as the Silver Processing DAG, is triggered automatically when new data becomes available in the Bronze layer. This is achieved using a FileSensor, which continuously monitors the directory for new files.

The use of Airflow provides several advantages, including task scheduling, dependency management, logging, and error handling. It also ensures that the pipeline is modular and easy to extend, allowing future integration of additional data sources or processing stages.

C. Layer Bronze Layer

All data ingested from the Reddit API and the web scraping process is stored as JSON files in the `datalake_bronze` directory. Each ingestion run generates a new file using a timestamp-based naming convention.

```
source_YYYYMMDD_HHMMSS.json
```

This naming strategy helps to ensure traceability and version control. Each file can be uniquely identified by its exact creation time, which facilitates debugging, auditing and re-processing if necessary. The Bronze layer provides the input for the Silver processing stage, ensuring that all subsequent

transformations are performed without altering the original data.

D. Silver Layer

The data in the Bronze layer is processed using tools such as Pandas. This includes various operations such as handling missing values, normalising data types, removing duplicates, detecting outliers and cleaning textual data for natural language processing (NLP).

Once processed, the data is stored in Parquet format using Apache Parquet. The files are saved in the `datalake_silver` directory, with names following this convention:

```
topic_YYYYMMDD_HHMMSS.parquet.
```

Parquet is chosen as the storage format due to its columnar structure, which significantly improves performance in analytical queries and reduces storage requirements thanks to efficient compression.

Furthermore, the Silver layer ensures traceability by maintaining a clear relationship between the processed data and its original input. This is achieved through consistent metadata fields and naming conventions, enabling each Parquet file to be linked to its corresponding JSON file.

E. Gold Layer

The Gold layer is the final stage of the data pipeline, where processed data is transformed into meaningful insights. While this layer is not yet implemented in the current phase of the project, it remains an integral part of the overall architecture.

At this stage, data from the Silver layer will be aggregated and enriched using advanced analytical techniques, such as sentiment analysis, topic modelling and trend detection. These processes generate refined datasets optimised for decision-making rather than raw analysis.

Implementing this layer may require scalable processing frameworks, such as PySpark, particularly if the dataset grows significantly in size.

F. Dashboard

The dashboard constitutes the visualization layer of the system, providing an interface for users to interact with the processed data. This component is planned to be implemented using Plotly Dash.

Through this interface, users will be able to explore sentiment trends, compare different data sources, and identify patterns in political and social discourse. The dashboard transforms complex datasets into intuitive visual representations, facilitating data-driven decision-making.

G. Docker environment

The entire process is implemented in a containerised environment using Docker and Docker Compose. This approach ensures that all components run in isolated and consistent environments, thereby eliminating dependency conflicts and simplifying deployment.

Each service in the process, including the Airflow components and the metadata database, runs in its own container. This modular configuration improves scalability and enables individual components to be updated or replaced without impacting the rest of the system.

H. Docker compose service definitions

The system is configured using a Docker Compose file (docker-compose.yml), which defines all the required services and how they interact. The main services include:

- Airflow Webserver: provides the graphical user interface for monitoring workflows
- Airflow Scheduler: responsible for executing tasks and managing dependencies
- Airflow Triggerer: handles asynchronous operations, such as sensors
- PostgreSQL Database: used as the metadata repository for Airflow

Docker volumes are configured to persist data outside the containers. This ensures that files stored in the Bronze and Silver layers are not lost when containers are restarted. Furthermore, mounting the DAGs directory as a volume allows for real-time updates without the need to rebuild the containers.

I. Technology Justification

The technologies selected for this project are based on efficiency, reliability and alignment with modern data engineering practices.

Apache Airflow was chosen because of its widespread industry adoption and its ability to manage complex workflows. Pandas and Polars were selected for their powerful data manipulation capabilities and ease of use in pre-processing tasks. Parquet is used as the storage format in the Silver layer due to its high performance in analytical workloads and efficient compression.

Docker is used to ensure reproducibility and simplify environment management, and request and beautiful soup is employed for web scraping thanks to its effectiveness in handling dynamic websites and anti-bot protections.

IV. PREPROCESSING PIPELINE DEFINITION

The preprocessing pipeline is one of the most critical stages of the project. It transforms heterogeneous raw textual data from the Bronze layer into a clean, structured dataset ready for analysis, which is then stored in the Silver layer.

As the project integrates data from the Reddit API and La Silla Vacía scraping, the preprocessing stage must address differences in schema, data quality, formatting and linguistic structure. The main objective of this stage is to standardise both sources into a unified format that supports natural language processing (NLP) and future sentiment analysis tasks.

This preprocessing pipeline is designed based on data samples collected and analysed in Workshop 1, where several

quality issues were identified, including missing values, duplicate records, inconsistent date formats, noisy text and highly variable engagement metrics.

A. Null Handling Strategy

One of the initial pre-processing steps involves identifying fields that may contain missing, incomplete or null values.

Based on the collected samples, the fields most commonly affected by missing data are:

Text, Author, Rating, Tags and Publication Date.

The handling strategy is defined according to the importance of each field within the analysis workflow. We will begin with:

Text field

This is one of the most important variables for sentiment analysis. However, some posts in the Reddit feed showed empty text values because the sentiment was fully expressed in the title. For this reason, the missing text values were not immediately removed. Instead, they were replaced with an empty string sentinel value.

”.”

This preserves the record and allows the title to remain available for analysis.

Author Field

In some extracted articles or deleted Reddit posts, the author field was unavailable. In these cases, a sentinel value is assigned:

’unknown_author’

This ensures schema consistency and allows for grouping operations if necessary.

Score field

The score field is numeric and represents engagement. If it is missing, it is imputed using the median score of the batch.

median(score).

The median is used instead of the mean because engagement values can be heavily skewed by viral posts.

Critical metadata fields

Fields such as date, source and title are mandatory. Records that do not include any of these fields are discarded, as they compromise traceability and temporal analysis. This enables data preservation with analytical reliability.

B. Data Type Normalization

As the data originates from various sources, several fields require normalisation to ensure consistency across the Silver layer.

The target data types are defined as follows:

Field	Data Type	Justification
title	string	Main textual content
text	string	NLP processing
author	category	Repeated low-cardinality values
date	datetime	Time-series analysis
source	category	Source comparison
score	float	Statistical analysis
tags	list/string	Topic classification

TABLE I
DATA TYPE

The date field requires special normalisation because Reddit provides Unix timestamps and scraped data provides ISO-style dates. Both are converted into a unified date and time format. This standardisation is essential for future trend analysis in the Gold layer. The author and source fields are converted to categorical types to optimise memory usage and improve analytical performance.

C. Text Preprocessing for NLP

This stage is important for the project because the ultimate goal is to analyse the sentiment of political and social discourse. The text cleaning pipeline is applied to both the title and the body of the text.

Lowercasing

To ensure consistency, all text is converted to lowercase. Words such as: 'petro', 'petro', 'PETRO' are treated as the same token. This reduces vocabulary sparsity.

Punctuation Removal

Punctuation marks, special characters, and unnecessary symbols are removed. For example if we have this sentence "What do you think about Petro?" is changed to "what do you think about Petro" This simplifies tokenization.

Stop Word Removal

The project's focus on Spanish discourse necessitates the removal of stopwords using a Spanish stopwords list. Such as "de", "la", "el", "y", "que", "con", "para". This improves the relevance of remaining tokens.

Tokenization

The cleaned text is split into tokens (individual words). Example: "gobierno petro crisis" becomes "gobierno", "petro", "crisis". This step is fundamental for downstream NLP tasks.

Language-Specific Normalization

Since the project is focused on Colombian discourse in Spanish, additional normalization is applied accent normalization, removal of duplicated spaces, unicode cleaning and emoji

removal (mainly Reddit). An example is: "política" becomes "politica"

D. Deduplication Strategy

Since the process runs daily, duplicate records may appear in multiple ingestion batches. Duplicates are removed using a composite key:

title + date + source

This criterion is appropriate because titles may be repeated, dates alone are not sufficient and the combination of both ensures uniqueness. This process prevents duplicate sentiment counts and biased trend analyses.

V. DOCKER STACK SETUP

To ensure reproducibility, portability and consistency across different environments, the entire data pipeline is deployed using a containerised architecture based on Docker and Docker Compose.

This approach enables all system components, including orchestration, data processing and storage, to run in isolated containers with controlled dependencies. Docker Compose defines and executes multiple services together as a cohesive system, thereby simplifying both development and deployment. The Docker stack is designed to support a fully functional Apache Airflow environment running locally, including all the necessary services, such as the web interface, scheduler, triggerer and metadata database.

A. Docker Stack Setup

The system is configured via a Docker Compose file (docker-compose.yml), which defines all the necessary services, volumes and network configurations for the pipeline.

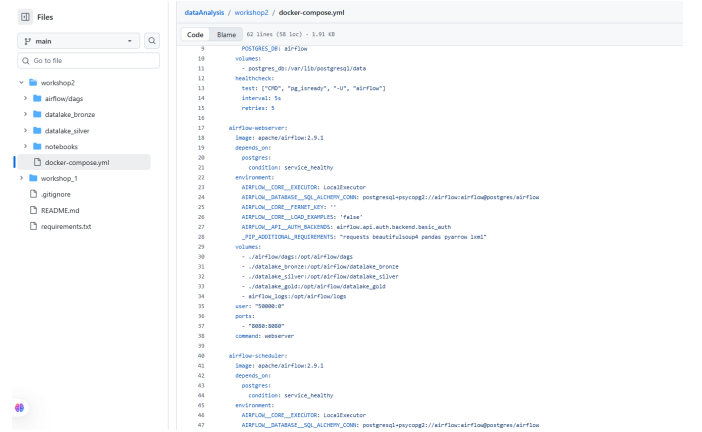


Fig. 2. docker-compose.yml

The architecture includes the following main components:

Airflow Webserver

This provides the graphical user interface (UI) for monitoring and managing workflows. It enables users to

visualise DAGs, trigger executions, inspect logs and track task statuses. This service is exposed locally and accessible through <http://localhost:8080>.

Airflow Scheduler

The scheduler is responsible for executing DAGs and managing task dependencies. It continuously monitors DAG definitions and triggers tasks according to their scheduling configuration (e.g., daily ingestion).

Airflow Triggerer

The triggerer manages asynchronous tasks, particularly those involving sensors, such as the FileSensor used in the Silver pipeline. This component manages long-running waiting operations to ensure efficient resource usage. PostgreSQL Metadata Database

PostgreSQL Metadata Database

Airflow requires a metadata database to store information about DAG runs, task states, logs, and execution history. In this project, PostgreSQL is used as the backend database due to its reliability and compatibility with Airflow.

B. Volume Configuration

To ensure data persistence and enable interaction between the host system and containers, several volumes are mounted in the Docker configuration.

The following directories are mapped:

datalake_bronze: Stores raw JSON files
datalake_silver: Stores processed Parquet files
datalake_gold: Reserved for future analytical data
airflow/dags: Contains DAG definitions

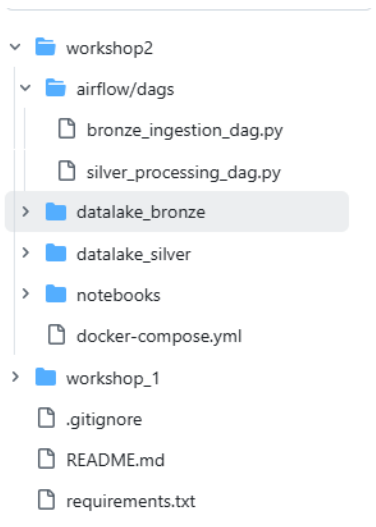


Fig. 3. Directories used for Docker

These volumes allow persistent storage of data outside containers, direct access to generated files from the host system

and live reloading of DAGs without restarting containers. Without these volumes, all generated data would be lost when containers are stopped.

C. Dependency Management

All Python dependencies required for the pipeline are defined in a requirements.txt file. These dependencies are installed within the Airflow environment to support data extraction, processing, and storage.

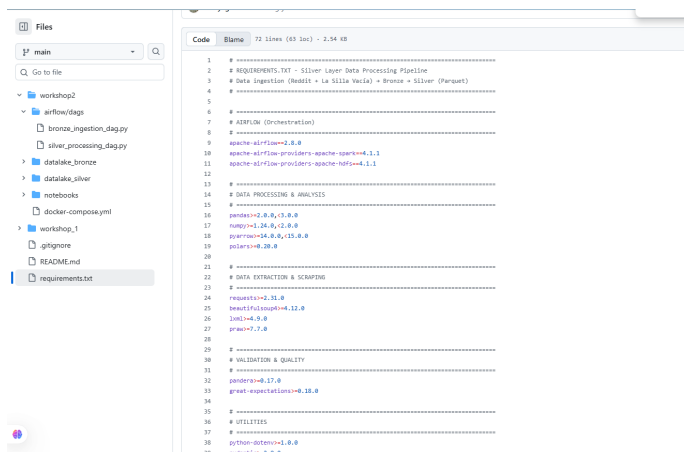


Fig. 4. requirements.txt

Key dependencies include:
requests → API communication
beautifulsoup4 → HTML parsing (scraping support)
pandas → data manipulation
polars → high-performance data processing
pyarrow → Parquet file handling
request and beautiful soup → browser automation for scraping

This approach ensures that all containers run with the exact same library versions, avoiding inconsistencies between environments.

D. Networking Configuration

Docker Compose automatically creates a shared network where all services can communicate using service names as hostnames. For example the Airflow services connect to PostgreSQL using the service name postgres. Internal communication between scheduler, webserver, and triggerer occurs within the same network. This eliminates the need for manual network configuration and ensures seamless communication between components.

E. Setup and Execution Process

The deployment process is straightforward and reproducible.

Step 1: Clone the Repository

The project repository is cloned from GitHub:

```
git clone https://github.com/Santayago710/dataAnalysis.git
cd dataAnalysis
```

Step 2: Initialize Airflow Environment

Airflow requires initialization of its metadata database:
docker-compose up airflow-init

Step 3: Launch All Services

All containers are started using:
docker-compose up

This command builds the containers (if necessary) and launches all services defined in the configuration.

Step 4: Access Airflow UI

Once the services are running, the Airflow interface is accessible at:

http://localhost:8080

Default credentials (if configured): username: airflow password: airflow

Step 5: Verify Execution

To confirm that the environment is correctly configured:

The Airflow UI should display the DAGs

The scheduler should be running

Tasks should be executable manually or automatically

Logs should be accessible

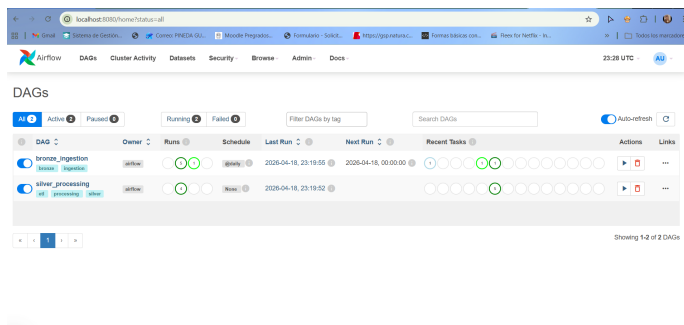


Fig. 5. Airflow IU with DaGs of the project

Additionally, the following command can be used to verify running containers:
docker ps

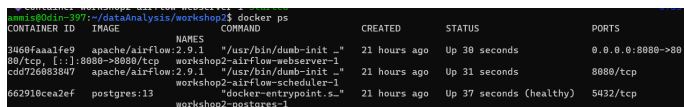


Fig. 6. Verification of active containers

Verification of Correct Setup

A successful setup is confirmed when all containers are running without errors, DAGs are visible and can be triggered from the UI, data is correctly written to datalake_bronze/ and datalake_silver and logs are generated and accessible. This ensures that the environment is fully operational and ready for pipeline execution.

VI. DAG IMPLEMENTATION AND DOCUMENTATION

The data pipeline is orchestrated using Apache Airflow, which automates, schedules and monitors all data ingestion and processing tasks.

Two main DAGs are implemented:

The Bronze Ingestion DAG is responsible for data extraction and raw storage, and the Silver Processing DAG is responsible for data preprocessing and transformation.

Both DAGs are implemented using the TaskFlow API, which allows workflows to be defined in a modular, readable and maintainable way. The pipeline follows a sequential dependency model: Data Sources, Bronze DAG, Bronze Layer, Silver DAG and Silver Layer.

A. Bronze Ingestion DAG

The Bronze Ingestion Data Augmentation Graph (DAG) extracts raw data from both data sources and stores it in the Bronze layer in JSON format. Its primary objective is to preserve the original data without transformation to ensure traceability and reproducibility.

DAG Configuration

DAG Name: bronze_ingestion_dag

Schedule: @daily (based on Workshop 1 ingestion strategy)

Catchup: disabled (to avoid backfilling unnecessary runs)

The daily schedule is chosen because political and social discourse evolves continuously but does not require real-time ingestion.

Tasks Definition

The DAG is composed of multiple tasks, each responsible for a specific step in the ingestion process.

1. API Data Extraction Task

This task connects to the Reddit API and retrieves recent posts from the r/Colombia subreddit. Sends HTTP requests using requests, parses JSON response and extracts relevant fields (title, text, author, date, score, url). At the output we obtain a list of structured Python dictionaries representing Reddit posts.

2. Web Scraping Extraction Task This task collects data from La Silla Vacía using browser automation. Uses request and beautiful soup to navigate pages, extracts article metadata and content, handles pagination and applies delays to avoid blocking. At the output we obtain a list of dictionaries representing scraped articles.

3. Data Validation Task This task performs a basic schema validation before storing the data. Required fields must exist (title, date, source), data must be in correct structure (list of dictionaries) and empty or malformed records are filtered. To prevent corrupt or invalid data from entering the Bronze layer.

4. JSON Persistence Task This task saves the validated data into the Bronze layer. File Naming Convention is:

source_YYYYMMDD_HHMMSS.json

Combines API and scraping data and generates timestamp writes JSON file to datalake_bronze. At the output we obtain

a new JSON file per DAG run.

Error Handling and Logging

Each task includes basic error handling mechanisms, such as try-except blocks for API requests and scraping, logging messages for successful and failed operations and graceful handling of empty responses. This ensures that failures are visible in the Airflow UI and do not crash the entire pipeline.

Expected Outputs

After each successful run a new JSON file is created in `datalake_bronze`, the file contains both Reddit and scraped data and the file is uniquely identified by timestamp.

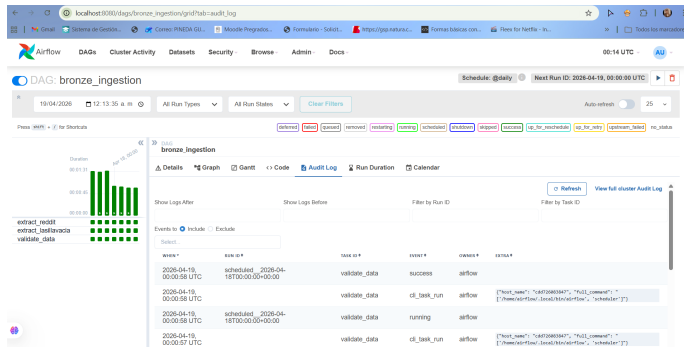


Fig. 7. Bronze DAG running successfully in Airflow

```
amias@odin-397:~/dataAnalysis/workshop2$ ls datalake_bronze
lasillavacia_20260406_230277.json  lasillavacia_20260418_022718.json  reddit_20260408_030731.json
lasillavacia_20260406_231111.json  lasillavacia_20260418_022952.json  reddit_20260408_032727.json
lasillavacia_20260406_232806.json  lasillavacia_20260418_023087.json  reddit_20260408_033457.json
lasillavacia_20260406_233081.json  lasillavacia_20260418_023211.json  reddit_20260408_033380.json
lasillavacia_20260406_233330.json  lasillavacia_20260418_023222.json  reddit_20260409_040196.json
lasillavacia_20260406_235806.json  lasillavacia_20260418_023330.json  reddit_20260409_040255.json
lasillavacia_20260407_000808.json  lasillavacia_20260418_231910.json  reddit_20260409_040608.json
lasillavacia_20260407_001806.json  lasillavacia_20260418_231957.json  reddit_20260409_041251.json
lasillavacia_20260407_002906.json  lasillavacia_20260419_000901.json  reddit_20260418_022633.json
lasillavacia_20260408_030731.json  reddit_20260408_030727.json  reddit_20260418_022718.json
lasillavacia_20260408_032727.json  reddit_20260418_022952.json  reddit_20260418_022952.json
lasillavacia_20260408_033457.json  reddit_20260418_023087.json  reddit_20260418_023087.json
lasillavacia_20260408_033380.json  reddit_20260418_023211.json  reddit_20260418_023211.json
lasillavacia_20260409_040196.json  reddit_20260418_023222.json  reddit_20260418_023222.json
lasillavacia_20260409_040255.json  reddit_20260418_023330.json  reddit_20260418_023330.json
lasillavacia_20260409_040608.json  reddit_20260418_231909.json  reddit_20260418_231909.json
lasillavacia_20260409_041251.json  reddit_20260418_231957.json  reddit_20260418_231957.json
lasillavacia_20260418_022633.json  reddit_20260407_000808.json  reddit_20260419_000901.json
```

Fig. 8. JSON files generated of Bronze DAG

The DAG is considered functional when it runs successfully in Airflow, all tasks complete without errors, and JSON files are generated in the Bronze folder.

B. Silver Processing DAG

The Silver Processing Dataflow Graph (DAG) transforms raw JSON data from the Bronze layer into clean, structured Parquet files, which are then stored in the Silver layer. This DAG implements the previously defined preprocessing pipeline, ensuring that the data is ready for analysis.

DAG Configuration

DAG Name: `silver_processing_dag`

Trigger Mechanism: FileSensor

Schedule: Event-based (triggered when new files appear)

Unlike the Bronze DAG, this DAG is not purely time-based. Instead, it reacts to new data availability.

FileSensor Mechanism

A FileSensor is used to monitor the `datalake_bronze` directory. It periodically checks for new JSON files. Uses parameters like `poke_interval`: Frequency of checks and `timeout`: Maximum waiting time. Once a new file is detected, the DAG proceeds to execution.

Tasks Definition

1. File Detection Task

Detects new JSON files in the Bronze layer. At output that we obtain is a path of the detected file.

2. Data Loading Task

Loads the JSON file into a DataFrame. Then, reads JSON and converts into Pandas/Polars structure.

3. Preprocessing Task

Applies the full preprocessing pipeline. Include null handling, data type normalization, text cleaning (lowercase, stopwords, tokenization), outlier detection and deduplication.

4. Parquet Persistence Task

Stores processed data in the Silver layer. The File Naming Convention is:

`topic_YYYYMMDD_HHMMSS.parquet`

Converts DataFrame to Parquet and saves in `datalake_silver`

Traceability Mechanism

Each Silver file maintains traceability to its corresponding Bronze file. This is achieved by including source filename as metadata, using consistent timestamps and maintaining batch-level processing. This ensures that every processed dataset can be traced back to its raw origin.

Expected Outputs

After execution we obtain a Parquet file is generated in `datalake_silver`, data is clean and structured and file is readable using Pandas or PySpark.

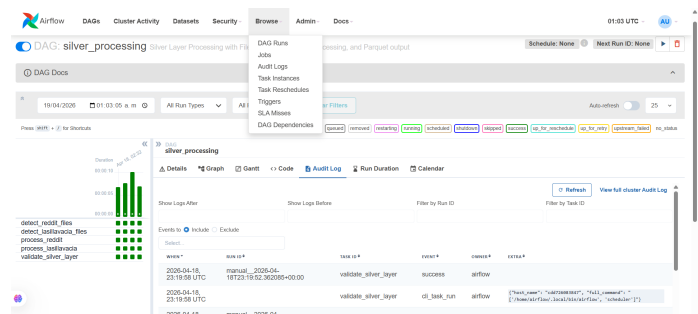


Fig. 9. Silver DAG running successfully in Airflow

```
lasillavacia_20260418_022633.json  reddit_20260418_022633.json  reddit_20260419_000901.json
amias@odin-397:~/dataAnalysis/workshop2$ ls datalake_silver
lasillavacia_20260406_040209.parquet  lasillavacia_20260418_023228.parquet  reddit_20260418_022955.parquet
lasillavacia_20260418_022637.parquet  lasillavacia_20260418_231912.parquet  reddit_20260418_023815.parquet
lasillavacia_20260418_022733.parquet  lasillavacia_20260418_231956.parquet  reddit_20260418_023207.parquet
lasillavacia_20260418_022955.parquet  reddit_20260409_042109.parquet  reddit_20260418_023228.parquet
lasillavacia_20260418_023014.parquet  reddit_20260418_022637.parquet  reddit_20260418_231912.parquet
lasillavacia_20260418_023207.parquet  reddit_20260418_022733.parquet  reddit_20260418_231956.parquet
amias@odin-397:~/dataAnalysis/workshop2$
```

Fig. 10. Parquet generated of Silver DAG

Verification

The DAG is considered successful when FileSensor detects new files correctly, DAG runs automaticall, Parquet files are generated and files can be read without errors.

VII. RESULTS EVIDENCE

This section presents the experimental evidence demonstrating the successful implementation and execution of the data pipeline. The results validate that both Airflow DAGs operate correctly, data is properly ingested into the Bronze layer, and the preprocessing pipeline generates structured outputs in the Silver layer. The evidence is organized following the pipeline flow, from orchestration to final data outputs.

A. Airflow DAG Execution

The first validation step consists of verifying that both DAGs are correctly deployed and executed in Apache Airflow (localhost:8080). As we can see, the process was carried out satisfactorily.

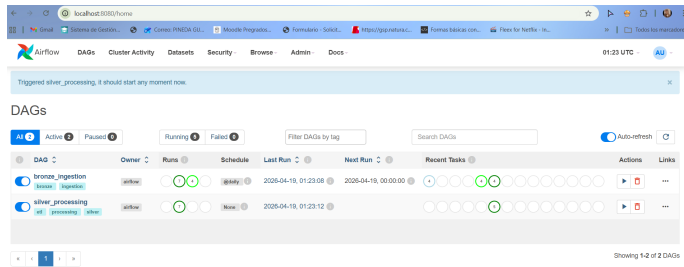


Fig. 11. Airflow IU running correctly with both DAGs

The figure 11 shows the main Apache Airflow interface, displaying both implemented DAGs. The visualisation confirms that bronze_ingestion_dag and silver_processing_dag have been successfully deployed and executed. The green status indicators show that all recent runs were completed without errors, which validates the correct configuration of the orchestration layer and confirms that the pipeline is operational.

B. Bronze DAG Execution

The Bronze DAG graph view illustrates the sequence of tasks involved in the ingestion process. Each task — API extraction, web scraping, validation and JSON persistence — is shown as having been successfully completed (in green). This confirms that the DAG is executing in the correct order and that the dependencies between the tasks are defined correctly. It also shows that data is consistently collected from both sources and stored without errors.

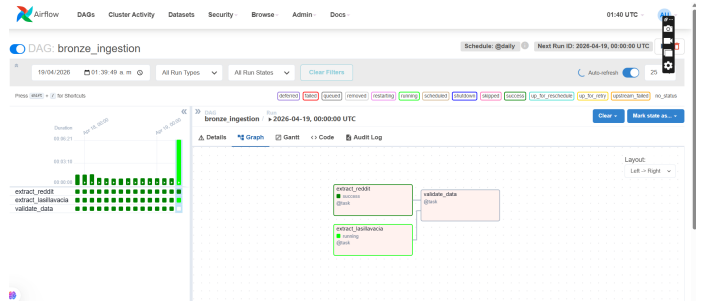


Fig. 12. Graph View of the Bronze DAG showing all completed tasks (green)

C. Silver DAG Execution

This figure shows how the Silver DAG is executed, including the FileSensor and subsequent preprocessing tasks. Successful execution of all nodes indicates that the system correctly detects new files in the Bronze layer, triggering the transformation process automatically. This validates the event-driven design of the pipeline, demonstrating that the preprocessing stage is fully integrated and functional.

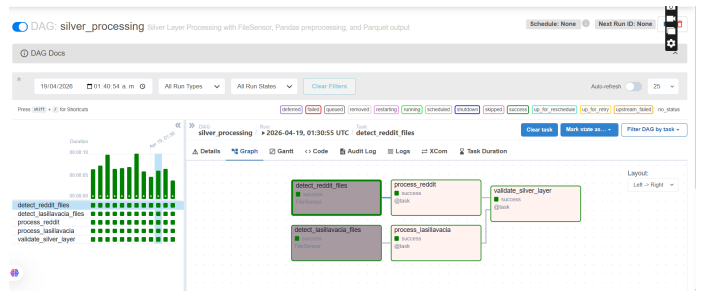


Fig. 13. Graph View of the Silver DAG showing all completed tasks (green)

D. Bronze Layer Outputs

The image 14 the contents of the 'datalake_bronze' directory, in which multiple JSON files are stored in accordance with the defined naming convention. Each file corresponds to a different ingestion run and is identified by a timestamp. This demonstrates that the pipeline is generating persistent raw data outputs and maintaining historical records, which are essential for traceability and reproducibility.

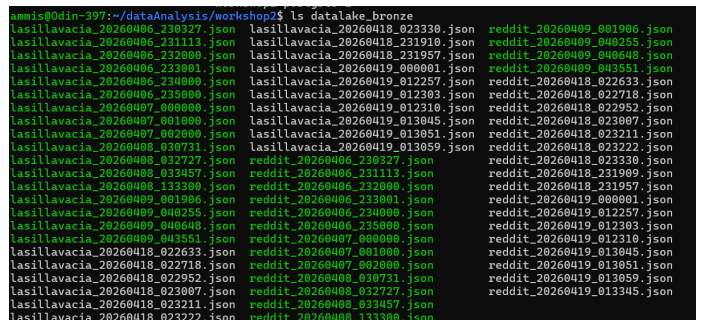


Fig. 14. JSON files generated by Bronze DAG

E. Sample JSON File

This figure shows the internal structure of a sample JSON file created using the Bronze DAG. The data includes fields such as 'title', 'text', 'author', 'date', 'source' and 'score', which confirms that information from both data sources has been successfully integrated. The fact that the data is in its raw format also demonstrates that no preprocessing has been applied at this stage, thus preserving the original content as intended in the Bronze layer.

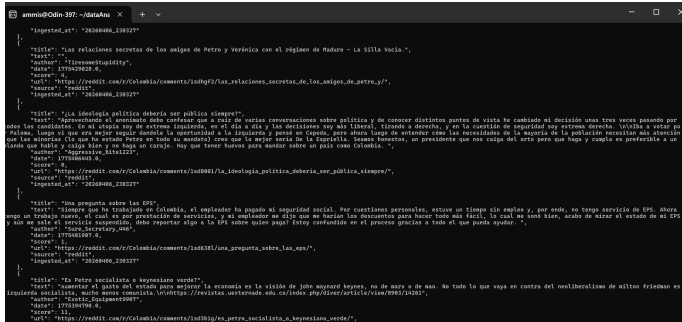


Fig. 15. Bronze DAG JSON Content

F. Silver Layer Outputs

The figure 16 illustrates the contents of the 'datalake_silver' directory, in which processed data is stored in Parquet format. The presence of multiple timestamp-based file names confirms that the Silver DAG is functioning correctly and generating structured outputs. These files represent cleaned and normalised datasets that are ready for analytical processing, thus validating the successful execution of the preprocessing pipeline.



Fig. 16. Parquets generated by Silver DAG

G. Parquet Data Inspection

Finally, this figure illustrates the inspection of a Parquet file created in the Silver layer following the preprocessing stage. The data has been loaded and displayed in the form of a DataFrame, which confirms that the file can be successfully read using tools such as Pandas. The visible columns and values show that the preprocessing pipeline has been applied correctly, including data type normalisation and text cleaning.

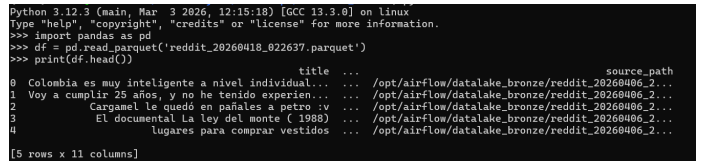


Fig. 17. Parquets content of Silver DAG

All previous evidence combined demonstrates that data is successfully extracted from multiple sources; that raw data is correctly stored in the Bronze layer; that the FileSensor automatically detects new data; that the preprocessing pipeline executes without error; and that clean data is stored in Parquet format in the Silver layer. This confirms that the pipeline operates correctly from ingestion to transformation.

VIII. CHALLENGES AND DECISIONS

During the development of the data flow, several technical and data-related challenges arose, requiring careful design decisions to ensure a robust and reliable system.

One of the main challenges was related to data extraction, particularly from La Silla Vacía. Unlike APIs, web scraping using requests and BeautifulSoup relies heavily on the HTML structure, which can be inconsistent or change over time. Additionally, some pages contained nested elements or irregular formatting, making it difficult to consistently extract fields such as article content, author, and publication date. To address this, flexible analysis strategies were implemented, including conditional extraction logic and validation checks to ensure that only complete and well-structured records were processed.

Another significant challenge was handling data quality issues from both sources. The Reddit API provided data with missing fields, especially in the text attribute, as some posts only contained titles. Similarly, inconsistencies in date formats and duplicate records were observed across multiple ingestion runs. These issues necessitated the implementation of a comprehensive preprocessing flow, including null handling strategies, data type normalization, and deduplication mechanisms to ensure consistency in the Silver layer. Pipeline orchestration using Apache Airflow also presented challenges, particularly in defining task dependencies and ensuring seamless communication between the Bronze and Silver stages. Properly configuring FileSensor required precise tuning of parameters such as polling intervals and timeouts to prevent unnecessary delays or file omissions. Separating ingestion and processing into two independent DAGs proved to be a key design decision that improved modularity and simplified debugging.

The containerization process with Docker presented additional difficulties. During setup, issues such as incorrect volume mounting, missing dependencies, and Airflow initialization errors were encountered. These were resolved by refining the docker-compose.yml configuration, correctly defining the shared volumes for data persistence, and ensuring that all necessary Python libraries were included in the environment.

IX. CONCLUSIONS

This project successfully demonstrates the design and implementation of an end-to-end data pipeline for ingesting and preprocessing political and social discourse data. The system captures both user-generated content and structured opinion articles by integrating heterogeneous sources, providing a richer analytical foundation.

Apache Airflow was used to automate and orchestrate the pipeline through modular DAGs, and the FileSensor mechanism demonstrated the effectiveness of event-driven processing. The layered Bronze and Silver architecture ensured data traceability and progressive refinement, preserving raw data while producing clean, structured outputs.

The adoption of Apache Parquet also demonstrated the importance of efficient storage for analytical workloads, while the use of Docker ensured reproducibility and consistency across environments.

The successful DAG executions, generation of JSON and Parquet files, and validation of the end-to-end pipeline confirm that the system operates correctly and meets its objectives. Furthermore, the current architecture leaves the system fully prepared for the implementation of the Gold layer, where advanced analytics such as sentiment analysis and data aggregation can be incorporated to generate actionable insights.

X.

REFERENCES

- [1] Reddit, “Reddit API Documentation,” [Online]. Available: <https://www.reddit.com/dev/api/>. [Accessed: Mar. 2026].
- [2] Reddit Inc., “Reddit Content Policy,” [Online]. Available: <https://www.redditinc.com/policies/content-policy>. [Accessed: Mar. 2026].
- [3] K. Reitz, “Requests: HTTP for Humans,” Python Software Foundation, [Online]. Available: <https://docs.python-requests.org/>. [Accessed: Mar. 2026].
- [4] L. Richardson, “Beautiful Soup Documentation,” [Online]. Available: <https://www.crummy.com/software/BeautifulSoup/>. [Accessed: Mar. 2026].
- [5] Microsoft, “Playwright Documentation,” [Online]. Available: <https://playwright.dev/python/docs/intro>. [Accessed: Mar. 2026].
- [6] W. McKinney, “pandas: Powerful Python data analysis toolkit,” [Online]. Available: <https://pandas.pydata.org/docs/>. [Accessed: Mar. 2026].
- [7] R. Vink, “Polars Documentation,” [Online]. Available: <https://polars.github.io/polars/>. [Accessed: Mar. 2026].
- [8] D. Crockford, “The JSON Data Interchange Format,” RFC 8259, 2017.
- [9] R. Mitchell, *Web Scraping with Python: Collecting More Data from the Modern Web*, 2nd ed. O’Reilly Media, 2018.
- [10] T. White, *Hadoop: The Definitive Guide*, 4th ed. O’Reilly Media, 2015.
- [11] Apache Software Foundation, “Apache Airflow Documentation,” [Online]. Available: <https://airflow.apache.org/docs/>. [Accessed: Mar. 2026].